

claude-windows / 96963dab-d38f-488c-af44-530ddbfeed1

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\96963dab-d38f-488c-af44-530ddbfeed1\subagents\workflows\wf_dd531439-5e0\agent-ab87602ffe55dc58a.jsonl`
- SHA-256: `64d782ea3a8e7af5b3ff1fae502d282eb50a987e1120c6a4e9a92c27d38d9c47`
- Source modified: `2026-07-08T06:36:06+00:00`
- Imported at: `2026-07-08T15:59:27+00:00`
- project: `wf_dd531439-5e0`
- session_id: `96963dab-d38f-488c-af44-530ddbfeed1`
```

Transcript

1. user (2026-07-08T06:26:58.680Z)

You are reviewing an UNCOMMITTED diff in an isolated git worktree at C:/Users/avidu/AppData/Local/Temp/claude/wt524-969 (branch feat/524-l4-primary, based on origin/master).

To see the change: cd to the worktree and run `git --no-pager diff` (working tree vs HEAD) and `git --no-pager diff --stat`; read the new files docs/COMPUTE_DECOUPLED_SERVING/L4_PRIMARY_TOPOLOGY.md and tests/test_video_id_trust.py directly. Read surrounding code as needed ? do NOT limit yourself to diff hunks.

CONTEXT ? GitHub issue #524 (epic): collapse the pipeline onto the L4 worker; shrink control-plane to a thin router. This PR delivers (1) a design doc resolving the compute-topology question (recommend co-located scale-to-zero Cloud Run Jobs + bucket upload; REJECT upload-onto-L4 and persistent-GPU with a cost model), and (2) "win #2": the control-plane drain trusts the SERVER-DERIVED submit-time md5 (jobs.video_id ? submit_time_video_id, #484 ? GCS metadata md5 converted to hex by backend/storage/gcs.py::md5_hex) instead of re-hashing the fetched clip, gated by deployment.trust_submit_video_id (default True) and guarded by orchestration._looks_like_video_id (strict 32-char lowercase hex). Win #1 (validation-on-L4) is deliberately NOT in this PR ? it is delivered by the separate open branch feat/521-compile-on-l4 (phase_placement.validation); duplicating it is forbidden.

Report ONLY concrete, actionable findings with file:line. No praise, no restating the design. For each: what breaks, the exact failing input/state, and the fix. If a dimension is clean, say so explicitly.

DIMENSION: DESIGN-DOC ACCURACY (docs/COMPUTE_DECOUPLED_SERVING/L4_PRIMARY_TOPOLOGY.md). This doc drives an owner decision, so every factual claim must be true. Verify against the ACTUAL code/runlog:

- Every [verified] tag: is the cited code anchor real and does it say what the doc claims? Spot-check submit_time_video_id (#484), the re-hash at orchestration, uploads.py streamed md5, #521's phase_placement (git show origin/feat/521-compile-on-l4:backend/config/placement.py), D4/D7/D16 in ./README.md.
- The cost model (\$4.2): the GCP rates ? are the numbers internally consistent (L4 \$0.00022404/s ? \$0.806/hr; g2-standard-4 \$0.872/hr ? ~\$637/mo; breakeven 51%)? Recompute. Flag any arithmetic error or misleading framing.
- The central claim: "uploading onto the L4 trades a ~free seconds-fast intra-region LAN pull for ~175s idle GPU". Is the idle-GPU reasoning sound (GPU must be up during the whole upload since compute needs the whole file)? Is the "same-region GCS?Cloud Run is free/LAN-fast" claim properly hedged as [researched] not [verified]?
- Does the doc HONESTLY represent that it recommends AGAINST the epic's proposed direction #1, and flag the #521-leaves-open gaps (validation-verdict serialization, #513 threshold forwarding)?
- Any broken internal markdown links (PHASE_PLACEMENT.md doesn't exist on master ? are those links external-URL or backtick, not relative)? Confirm check_md_links would pass.
- Does \$5/\$7 correctly credit win #1 to #521 and NOT claim to implement it here?

2. assistant (2026-07-08T06:27:02.026Z)

I'll start by examining the diff and the design doc in the worktree.

3. user (2026-07-08T06:27:04.569Z)

warning: in the working copy of 'docs/COMPUTE_DECOUPLED_SERVING/L4_PRIMARY_TOPOLOGY.md', LF will be replaced by CRLF the next time Git touches it

warning: in the working copy of 'tests/test_video_id_trust.py', LF will be replaced by CRLF the next time Git touches it

```
backend/api/job_worker.py          | 5 +
backend/config/models.py           | 10 +
backend/orchestration.py           | 51 ++++
.../L4_PRIMARY_TOPOLOGY.md         | 225 +++++
docs/COMPUTE_DECOUPLED_SERVING/README.md | 2 +
tests/test_config_deployment.py     | 2 +
tests/test_video_id_trust.py        | 173 +++++
7 files changed, 466 insertions(+), 2 deletions(-)
```

4. user (2026-07-08T06:27:05.805Z)

bf97700 docs(runlogs): record live SPA CUJ closing the 2 owner-gated validation-latency cells (#522)

699d85b feat(validation): cache validation verdict by video_id to skip re-validation on reupload (#518)

71c88ef docs(runlogs): validation-latency closure report for #512 + #513 (cloud-serving) (#519)

115b83d docs(runlogs): record live smoke of the crash-before-marker fast-fail (#515) (#517)

ef41473 fix(cloud-run): fail fast when a worker execution terminates without a done-marker (#515)

5. user (2026-07-08T06:27:12.518Z)

```
diff --git a/backend/api/job_worker.py b/backend/api/job_worker.py
index a90b14e..f8da26b 100644
--- a/backend/api/job_worker.py
+++ b/backend/api/job_worker.py
@@ -173,6 +173,11 @@ def _run_claimed_job(job: Dict[str, Any], config: Any, db: Any) -> None:
     db,
     req,
     progress=lambda stage: db.set_job_progress(job_id, stage),
+    # #524 win #2: hand the drain the submit-time md5 already on the job row
+    # (`submit_time_video_id`, #484 ? server-derived, byte-identical to
+    # compute_video_id) so it can skip re-hashing the fetched clip. "" ? unknown
+    # (local upload / non-gcs URI) ? the drain hashes as before.
+    trusted_video_id=job.get("video_id") or None,
 )
 if result.get("video_id"):
     db.set_job_video_id(job_id, result["video_id"])
diff --git a/backend/config/models.py b/backend/config/models.py
index a7c997c..71e43b8 100644
--- a/backend/config/models.py
+++ b/backend/config/models.py
@@ -481,6 +481,16 @@ class DeploymentConfig(BaseModel):
     # 1800s ExecutionPolicy default undercut real 1080p60 jobs (~22 min detect + queue/cold-start,
     # observed on the 2026-07-05 owner CUJ) and could abandon a slow-but-healthy paid GPU run.
     remote_poll_max_wait_sec: float = Field(default=3900.0, gt=0)
+    # Trust the SUBMIT-TIME md5 instead of re-hashing the fetched file on the drain (#524 win #2).
+    # The chunked upload already streams the whole-file md5 (`backend/api/uploads.py`), and
+    # the
+    # submit hook records it on the job row (`submit_time_video_id`, one gs:// metadata stat
+    # ? #484);
+    # that value is BYTE-IDENTICAL to `compute_video_id` (same bytes, same MD5) and is
```

```

ALREADY
+ # trusted for the submit-time cache probe (``backend/api/jobs.py``). When True (default)
the drain
+ # reuses it and SKIPS re-hashing the multi-GB clip it just fetched to scratch; when the id
is absent
+ # (a local/appliance upload or a non-gcs direct-URI submit ? ``submit_time_video_id``
returns None)
+ # the drain still hashes, exactly as before. Set False to force a re-hash unconditionally
+ # (belt-and-suspenders rollback ? reverts to pre-#524 behaviour with no redeploy).
+ trust_submit_video_id: bool = True

class BillingConfig(BaseModel):
diff --git a/backend/orchestration.py b/backend/orchestration.py
index 8b7851e..7769e4e 100644
--- a/backend/orchestration.py
+++ b/backend/orchestration.py
@@ -239,6 +239,20 @@ def submit_time_video_id(input_ref, cfg) -> Optional[str]:
    return None

+def _looks_like_video_id(value: Optional[str]) -> bool:
+    """True iff ``value`` is a well-formed ``video_id`` ? the 32-char lowercase hex MD5 that
+    ``compute_video_id`` produces (``backend/utils/hashing.py``). Guards the #524 win-#2 trust
path:
+    a trusted id is only reused in place of a re-hash when it has the EXACT shape the drain
would
+    have computed, so a malformed / truncated / wrong-encoding id can never become the DB key ?
+    it degrades to a real hash instead. ``submit_time_video_id`` already normalizes the gs://
+    base64 md5 to this hex form (#484, ``backend/storage/gcs.py::_md5_hex``)."""
+    return (
+        isinstance(value, str)
+        and len(value) == 32
+        and all(c in "0123456789abcdef" for c in value)
+    )
+
def _cached_process_payload(
    video_id: str, validation_results, segments, requested_compute
) -> Dict[str, Any]:
@@ -706,7 +720,7 @@ def _maybe_dispatch_remote(

def _execute_processing(
-    config=None, db=None, req=None, progress=None
+    config=None, db=None, req=None, progress=None, trusted_video_id=None
) -> Dict[str, Any]:
    """The full hash ? validate ? segment ? report pipeline for one video.

@@ -716,6 +730,15 @@ def _execute_processing(
    test code that calls ``_execute_processing(req)`` still works.

    ``progress`` is an optional callable(stage: str) used to surface coarse job progress.
+
+    ``trusted_video_id`` is the SERVER-DERIVED submit-time md5 the drain already knows for this
+    job (``jobs.video_id`` ? ``submit_time_video_id``, #484) ? byte-identical to
+    ``compute_video_id`` and already trusted for the submit-time cache probe. When present (and
+    ``deployment.trust_submit_video_id`` is on), it is reused so the drain SKIPS re-hashing the
+    multi-GB clip it just fetched (#524 win #2); absent / malformed ? the drain hashes as
before.
+    It is a keyword-only, internal channel ? NOT a client-supplied request field (a caller's
+    ``/api/process`` body cannot inject it; the submit hook recomputes the authoritative id).
+
+    Raises on unexpected internal errors ? the caller records those as a job failure
    """

```

```

import backend.main as _main # call-time seam: main's globals/model stay patchable
@@ -748,7 +771,31 @@ def _execute_processing(
    local_video = storage.acquire_local_input(
        req.video_path, getattr(config, "storage", None)
    )
-    video_id = _main.compute_video_id(local_video)
+    # #524 win #2: trust the SERVER-DERIVED submit-time md5 instead of re-hashing the whole
+    # (often multi-GB) clip we just fetched. The trusted id is byte-identical to
+    # compute_video_id (same bytes, same MD5 ? #484) and is ALREADY the value the submit-
time
+    # cache probe trusts, so reusing it here is consistent, not a new trust assumption.
Gated by
+    # deployment.trust_submit_video_id (default on) and only honoured when the id is well-
formed
+    # (_looks_like_video_id) ? any absence/malformation degrades to a real hash. The fetch
above
+    # is unchanged; only the redundant hash is skipped.
+    trust_submitted = bool(
+        getattr(
+            getattr(config, "deployment", None), "trust_submit_video_id", True
+        )
+    )
+    # STRICT: trust only an id in the EXACT form the drain would have produced (32-char
+    # lowercase hex, per compute_video_id / submit_time_video_id). No normalization ? a
+    # differently-cased or malformed id is a signal something is off, so re-hash rather
than
+    # silently coerce it into the DB key.
+    candidate = trusted_video_id or ""
+    if trust_submitted and _looks_like_video_id(candidate):
+        video_id = candidate
+        logger.info(
+            "[API] trusting submit-time video_id=%s ? skipped the drain re-hash (#524).",
+            video_id,
+        )
+    else:
+        video_id = _main.compute_video_id(local_video)
except Exception:
    storage.cleanup_acquired_local_input(
        req.video_path, local_video, getattr(config, "storage", None)
diff --git a/docs/COMPUTE_DECOUPLED_SERVING/README.md b/docs/COMPUTE_DECOUPLED_SERVING/README.md
index e60d644..d0f3fea 100644
--- a/docs/COMPUTE_DECOUPLED_SERVING/README.md
+++ b/docs/COMPUTE_DECOUPLED_SERVING/README.md
@@ -11,6 +11,8 @@

```

> **Northstar (D14):** a **mobile, app-like** flow ? point at a **Google Drive** video ? **fully cloud-native** run ? the stitched, **ready-to-share** reel lands **back in Drive**, next to the source. Two design promises around it: a user can **switch local?cloud anytime** (D13, a headline feature), and execution is **never a surprise** ? auto-detect **suggests**, the user **confirms** (D15).

+> **#524 (2026-07-08):** the **L4-primary topology** question ? "collapse upload?stitch onto the worker, shrink the control-plane to a thin router" ? is resolved in [L4_PRIMARY_TOPOLOGY.md](./L4_PRIMARY_TOPOLOGY.md). It **recommends against** uploading directly onto the GPU box (a cost model shows that trades a ~free intra-region LAN pull for minutes of idle GPU) and instead **collapses validate+detect+stitch** onto the scale-to-zero L4 Job via #521's **phase_placement** ? honoring D4/D7/D16. Ships **win #2** here (drop the redundant control-plane re-hash; trust the submit-time md5, reversible).

0. TL;DR

The rally engine is a **pure function** of (input bytes + config) ? outputs in three deterministic stages ? **DETECT** ? **WINDOW** ? **STITCH** ? that **no deployment profile may branch on**. A **ComputeBackend** (deployment profile) decides **where** the core runs (a local process vs a Cloud Run GPU job); a **StorageBackend** decides **where bytes come from** (local FS / USB / mounted Google Drive / bucket). **One startup config** picks both. We ship **truly-local**

(local GPU + in-process stitch, zero cloud) and **cloud-serving** (upload <2GB / gdrive / bucket ? a Cloud Run GPU job runs detect **and** stitch, so stitch's compute is metered), plus **cpu-mock** for CI. The single most important caveat: detection is **byte-identical** only on the same GPU (cross-GPU is sub-pixel-different, per the 2026-06-20 deploy report) ? so the **parity guarantee** is enforced at the WINDOW + STITCH layer, not detector-CSV bytes.

```
diff --git a/tests/test_config_deployment.py b/tests/test_config_deployment.py
index 1e4cfa6..48150a2 100644
```

```
--- a/tests/test_config_deployment.py
```

```
+++ b/tests/test_config_deployment.py
```

```
@@ -70,6 +70,7 @@ def test_default_config_dump_only_adds_deployment_section():
```

```
    "compute_target": "",
```

```
    "max_concurrent_remote_jobs": 4,
```

```
    "remote_poll_max_wait_sec": 3900.0,
```

```
+    "trust_submit_video_id": True,
```

```
    }
```

```
@@ -397,6 +398,7 @@ def test_save_default_config_includes_deployment(tmp_path):
```

```
    "compute_target": "",
```

```
    "max_concurrent_remote_jobs": 4,
```

```
    "remote_poll_max_wait_sec": 3900.0,
```

```
+    "trust_submit_video_id": True,
```

```
    }
```

```
    cfg = load_config(str(p))
```

```
    assert cfg.deployment.profile == ""
```

6. user (2026-07-08T06:27:12.849Z)

1 # L4-primary pipeline topology ? collapse the pipeline onto the worker, shrink the control-plane to a thin router

2

3 > **Status:** `DRAFT` ? **owner-review** (recommends **against** the epic's proposed direction #1; see §0/§8) · **Owner:** Avi · **Created:** 2026-07-08 · **Last updated:** 2026-07-08

4 > **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when DONE)

5 > **Issue:** [#524](https://github.com/Khelsutra/badminton-highlight-indexer/issues/524) (epic) · **Builds on:** [#521](https://github.com/Khelsutra/badminton-highlight-indexer/issues/521) (`phase\_placement`) · **Motivated by:** [#520](https://github.com/Khelsutra/badminton-highlight-indexer/issues/520) (the tracing that surfaced the data-movement finding)

6 > **Relation to existing docs:** extends [COMPUTE_DECOUPLED_SERVING/README.md](./README.md) (ADR-014) and [PHASE_PLACEMENT.md](#521)](https://github.com/Khelsutra/badminton-highlight-indexer/blob/feat/521-compile-on-l4/docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md); it does **not** supersede them ? it answers the **topology** question they left open and sequences the migration on top of `phase\_placement`.

7 > **Honesty note:** each claim is tagged `[verified]` (checked against code/runlog), `[researched]` (needs sign-off / external pricing I could not fetch live), or `[design]` (proposed). Not legal/financial advice.

8

9 ---

10

11 ## 0. TL;DR

12

13 The 2026-07-08 live CUJ moved one 1.83 GiB clip **3x** (upload?bucket, bucket?control-plane to validate, bucket?worker to detect) and pinned the CPU-only 2-vCPU control-plane for **~16 min** of heavy work (validation 183 s + stitch ~12m55s). The epic proposes fixing this by **uploading directly onto the L4** and async-copying to the bucket. **This design recommends against** that direction and quantifies why: a same-region bucket?compute transfer inside GCP is **LAN-fast** and not billed as internet egress **[researched]**, so the round-trip that upload-onto-L4 saves is worth **~seconds** and **~\$0**, while keeping an L4 up to **receive** a **minutes-long** upload burns **~175 s** of idle GPU per run (**~\$0.08±0.16**) **[design]** and **opens** a durability exposure window instead of closing one.

14

15 The cheaper, more durable, already-ratified path is the opposite: ****keep the upload streaming to the cheap bucket**** (the #480 design ? the durable copy exists ***before*** the L4 ever starts), and ****collapse validate + detect + stitch onto the scale-to-zero Cloud Run L4 Job**** ? which is exactly what ****#521's `phase\_placement` already wires as a config flip****. The residual **"~30 s pre-validation gap"** is then killed not by moving the ***upload*** but by the control-plane ****never downloading the file at all**** (it delegates validation to the L4 per #521, and trusts the already-known submit-time md5 per this epic's win #2 ? no re-hash). Net data movement drops from 3x to the theoretical floor: ****1 client upload + 1 LAN-fast intra-region pull****, with ****zero idle-GPU cost**** and ****zero exposure window****.

16

17 ****Actionable now (this PR):**** the design doc + ****win #2**** (drop the redundant control-plane re-hash, config-guarded, reversible). ****Win #1**** (validation-on-L4) is ****delivered by #521**** ? this doc does not duplicate its knob. A persistent GPU (GCE VM / Cloud Run Service-with-GPU) is ****rejected at current volume**** on a quantified cost basis and left as an owner-gated future ADR.

18

19 **## 1. Problem & goal**

20

21 ****Why now.**** Tracing the live CUJ (``runlogs/DEPLOY_REPORT_cloud-serving_validation-latency_2026-07-07.md`` §9 ``[verified]``) produced two findings:

22

23 1. ****Data is moved 3x per run**** ``[verified]``: (a) upload streams straight to ``gs://?/uploads/?`` (never touches control-plane disk ? the #480/#471 OOM fix, ``backend/api/uploads.py``), computing the md5 incrementally as it streams; (b) the control-plane re-downloads the whole clip to scratch to validate ? ``_execute_processing`` ? ``storage.acquire_local_input(gs://?)`` at ``backend/orchestration.py:748``, then a ****redundant re-hash**** ``compute_video_id`` at ``:751``; (c) the worker downloads it a third time to detect (``[worker]`` pulled ?). The unlogged ****~30 s pre-validation gap**** is finding (b).

24 2. ****The control-plane is the scaling wall**** ``[verified]``: validation (183 s) ****and**** stitch (~12m55s, ~67 s/clip 4K re-encode, no NVENC) both run in-process on the 2-vCPU control-plane, serialized by ``_LOCAL_COMPUTE_LANE`` ? ~16 min of heavy CPU per run ? the API saturates past ~5 concurrent users.

25

26 ****The concrete outcome we want.**** The L4 worker becomes the primary compute for the whole pipeline; the control-plane shrinks to a ****thin router**** (auth · dispatch · DB · SPA + job status · signed-URL download) that does ****no heavy decode/encode**** and, ideally, ****never pulls the full file****. Every move must be ****reversible by config**** with a ****bucket-mediated fallback****, and land as small PRs on top of ``origin/master``.

27

28 ****The hard question this doc must resolve (compute topology).**** "Upload directly to the L4" needs an ****inbound endpoint on the GPU box****, but the current L4 is a Cloud Run ****Job**** (batch, no inbound HTTP ? ``backend/compute/cloud_run.py``, dispatched per-request, ``[verified]``). So the epic forces a choice between ****A. Cloud Run Service-with-GPU****, ****B. GCE L4 VM****, and ****C. a hybrid**** presign/redirect. §4 answers it with numbers.

29

30 ****Read to get current:**** ``[PHASE_PLACEMENT.md`` (#521)](https://github.com/Khelsutra/badminton-highlight-indexer/blob/feat/521-compile-on-l4/docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md) ? the ``phase_placement`` knob this sequences on, ``[README.md]``(``./README.md``) §2 decisions ****D4/D7/D16**** (the ratified constraints that bound the options), and the runlog §9 (the evidence).

31

32 **## 2. Decisions locked**

33

#	Decision	Source / date	Implication
L1	**Keep upload ? bucket** (streaming, #480). The durable copy exists **before** any compute runs; the bucket *is* the "reliable co-located disk" the epic wants ? no VM needed for durability. this doc <code>`[design]`</code> + #480 <code>`[verified]`</code> No exposure window; upload cost stays on the cheap I/O path.		
L2	**Collapse validate + detect + stitch onto the scale-to-zero Cloud Run L4 Job** via <code>`phase_placement`</code> (#521), not onto a persistent GPU. Honors **D7** (scale-to-zero) + **D16(a)** (Jobs, not Services). #521 <code>`[verified]`</code> + D7/D16 The "L4-primary" vision is realized by **config flips** , not a rearchitecture.		
L3	**Kill the ~30 s gap by *not downloading to the control-plane*, not by uploading		

onto the L4.** Compose #521's validation-delegation (CP skips the validator suite) with win #2 (CP trusts the submit-time md5 ? no re-hash). | this doc `[design]` | The CP stops pulling the full file when validation is delegated; movement ? 3x2x. |

39 | L4 | **Reject persistent-GPU (GCE VM / Service-with-GPU) at current volume.**
 Breakeven vs scale-to-zero is a **~51 % on-demand / ~31 % spot duty cycle** (? 2,600?4,300 runs/mo sustained) `[design]`; we are at pilot volume. Revisit only behind a **new ADR** at sustained load. | §4.2 cost model `[design]` | Preserves per-run billing (D7/D8); avoids ~\$382?637/mo idle. |

40 | L5 | **Reject "upload onto the L4."** It saves a ~free, seconds-fast intra-region LAN pull at the cost of ~175 s idle GPU/run **and** opens a durability window. | §4.3 `[design]` | The epic's proposed direction #1 is not adopted; owner sign-off requested (§8). |

41 | L6 | **Win #2 (drop the re-hash) is config-guarded + reversible**
 (`deployment.trust\_submit\_video\_id`, default on) and produces a **byte-identical `video\_id`** (the GCS `md5Hash` over the same bytes = `compute\_video\_id`). | win #2 `[verified]` | Zero-regression; one-flag rollback. |

42

43 *(IDs are `L#` to avoid colliding with README.md's `D#`. These refine ? never override ? D4/D7/D16.)*

44

45 ## 3. Foundation ? the ratified constraints this must live inside

46

47 The topology is **not a blank slate** ? ADR-014 (README.md §2) already ratified the serving substrate. The options in §4 are scored **against** these, and any option that breaks one needs a new ADR, not just this doc:

48

49 | Ratified decision | What it pins | Consequence for #524 |

50 |---|---|---|

51 | **D4** | In cloud-serving, **stitch runs on Cloud Run** (co-located, metered). | The live system's CP-stitch is a *deviation* from D4; #521 *realizes* D4. #524 is mostly **completing an already-ratified decision**, not inventing one. |

52 | **D7** | **Scale-to-zero, no warm pool** (a ~\$500/mo idle GPU "would break per-video billing"). | Any persistent-GPU option (B, Service-with-min-instances) **contradicts D7** ? new-ADR territory (§4.2). |

53 | **D16(a)** | **Cloud Run Jobs** (not Services-with-GPU), **bucket-poll** for completion. | Option A (Service-with-GPU inbound upload) **contradicts D16(a)**. The Job model is load-bearing (reuses `JobWaiting`/`claim\_due\_job`). |

54 | **D16(b)** | Stitch encode = libx264 for parity; **NVENC deferred behind a knob**. | #521 lands exactly that knob (`cloud\_stitch\_encoder`), using this CUJ as the cost data D16(b) said to revisit on. |

55 | **D16(c)** | Input cap hard-reject `< 2 GB`. | The 1.83 GiB CUJ clip is *at* the cap; the topology must stay sane at 2 GB (upload minutes, pull seconds). |

56

57 **Key reframing this foundation gives us:** the owner's "reliable persistent disk" requirement is *already met* by the **regional GCS bucket** ? a durable, co-located, cheap "disk" the upload already writes to directly. The persistent-VM instinct solves a problem (durable local disk) that the bucket-first design doesn't have.

58

59 ## 4. Design / architecture ? resolving the topology question

60

61 ### 4.1 The measured pipeline (evidence base) `[verified]`

62

63 From runlog §9 (rev `rally-control-plane-00019`, "First CUJ.MP4", 1.83 GiB / 174.17 s / 4K-class):

64

65 | Phase | Wall | Machine | GPU needed? | Notes |

66 |---|---|---|---|---|

67 | Upload (client ? bucket) | **~2m55s** | client uplink | no | 1.83 GiB @ ~84 Mbps ? **owner bandwidth**, not infra |

68 | Accept ? validate start | ~30 s | control-plane | no | **the gap** = CP re-download to scratch (`acquire\_local\_input` + re-hash) |

69 | Validation (#512 one-pass) | **~183 s** | control-plane 2 vCPU | no (CPU-bound) | scene_cut/blur/relevance shared decode |

70 | Dispatch ? worker ready | ~55 s | Cloud Run Job | ? | image pull + GCSFuse + torch/cuda init (cold start) |

71 | Detection (WASB) | **~161.6 s** | L4 (cuda) | **yes** | the only genuinely GPU-bound

```

phase |
72      | Ingest + finalize | ~4 s | control-plane | no | intervals ? DB |
73      | Compile / stitch | **~12m55s** | control-plane 2 vCPU | no (NVENC would need GPU) |
11x 4K re-encode+watermark |
74
75      **Two facts that drive the whole design:**
76      - **Only *detection* is GPU-bound.** Validation is CPU/decode-bound; stitch is encode-
bound (NVENC *wants* a GPU but libx264 doesn't *need* one). So "collapse onto the L4" is about
**co-locating CPU work next to the GPU on a box that's already up and already holding the
bytes**, not about the GPU per se.
77      - **Upload dwarfs compute** (175 s upload vs 161.6 s GPU-detect). Anything that keeps a
GPU allocated *during upload* roughly **doubles** the GPU-allocated time for zero compute.
78
79      ### 4.2 Cost model (asia-southeast1, USD, Cloud Billing Catalog API, 2026-07-08)
`[verified]` rates / `[design]` per-run
80
81      Rates pulled live from the Catalog API (`cloudbilling.googleapis.com`, project `gen-
lang-client-0306026826`):
82
83      | Resource | Rate | Source |
84      |---|---|---|
85      | Cloud Run **L4 GPU** (no zonal redundancy) | **$0.00022404 / s** = $0.806/hr | Catalog
`[verified]` |
86      | Cloud Run vCPU (instance/jobs) | $0.0000216 / s | Catalog `[verified]` |
87      | Cloud Run memory | $0.0000024 / GiB·s | Catalog `[verified]` |
88      | ? **L4 Job @ 8 vCPU / 32 GiB (running)** | **$1.705 / hr** ($0.000474/s) | derived
`[design]` |
89      | GCE **g2-standard-4** (4 vCPU/16 GiB + L4), on-demand | **$0.872 / hr** = **~$637 /
mo** | Catalog `[verified]` |
90      | GCE g2-standard-4, **spot/preemptible** | $0.523 / hr = ~$382 / mo | Catalog
`[verified]` |
91      | Cloud Run **GPU Service, warm** (min-instances=1, 4 vCPU/16 GiB) | ? $1.256 / hr =
**~$917 / mo** idle | derived `[design]` |
92      | Balanced PD | $0.11 / GiB·mo | Catalog `[verified]` |
93
94      **Per-run GPU-allocated cost (the term that varies by topology):**
95
96      | Topology | GPU-allocated time / run | GPU cost / run | ? vs today | Concurrency wall?
|
97      |---|---|---|---|
98      | **Today** (detect-only on L4; validate+stitch on CP) | ~222 s (cold+detect+io) |
**~$0.105** | ? | **Yes** ? ~16 min CP CPU/run |
99      | **#521 + win #2** (validate+detect+stitch on L4; upload?bucket) | ~312 s (validate 60
+ detect 162 + stitch 60 + io 30) | **~$0.148** | **+$0.04** | **No** ? CP is thin |
100     | **Upload-onto-L4** (GPU up during upload too) | ~487 s (+175 s idle upload) |
**~$0.231** | **+$0.13** | No, but idle-GPU tax |
101     | **Persistent GCE VM** (on-demand) | billed 24/7 regardless | **$637/mo flat** | breaks
per-run billing | No |
102
103     **Breakeven (persistent vs scale-to-zero):** a persistent on-demand g2 ($0.872/hr) beats
the scale-to-zero Job ($1.705/hr *while running*) only at a **duty cycle ? $0.872/$1.705 ? 51
%** (spot: ? 31 %). At ~312 s/run that is **~2,600?4,300 runs/month sustained, 24/7**
`[design]`. Pilot volume is *handfuls/day*. **Scale-to-zero wins by ~2 orders of magnitude** and
preserves per-run billing (D7/D8). A persistent GPU is also **idle >95 % of the time** at our
volume ? the ~$382?637/mo would be almost pure waste.
104
105     ### 4.3 The three topology options, scored
106
107     **Option A ? Cloud Run Service-with-GPU hosting upload + pipeline.**
108     - *Latency:* scale-to-zero ? same ~55 s cold start as the Job; **min-instances=1**
removes cold start but pins a **~$917/mo warm idle GPU**.
109     - *Cost:* to accept the upload, the GPU Service instance must be **up for the whole 175
s upload** ? the idle-GPU tax of $4.2 on *every* request, plus the warm-idle bill if min-
instances>0.
110     - *Ops/parity:* **contradicts D16(a)** (Jobs, not Services) and **D7** (scale-to-zero) ?

```

abandons the `JobWaiting`/`claim\_due\_job`/bucket-poll machinery that's shipped and tested. A Service also needs request-timeout/concurrency tuning the batch Job sidesteps.

111 - **Verdict: reject.** Buys an inbound endpoint we don't need (the bucket is the inbound endpoint) at the cost of the idle-GPU tax + two ratified-decision reversals.

112

113 **Option B ? GCE L4 VM (persistent, reliable PD).**

114 - **Reliability:** real, attached, reliable PD ? the owner's stated appeal. But **the bucket already gives us durable co-located storage** (L1), so the PD solves a non-problem.

115 - **Cost:** **\$382?637/mo flat**, >95 % idle at pilot volume; breaks per-run billing (D7/D8). Spot is cheaper but **preemptible** ? directly against the "reliable" motivation.

116 - **Ops:** we now **own a VM** ? patching, disk-fill, crash-recovery, autoscaling-group if we ever need >1. The Job model has none of this.

117 - **Verdict: reject** at current volume; owner-gated future ADR if sustained load crosses the \$4.2 breakeven.

118

119 **Option C ? Hybrid: control-plane presigns/redirects the upload to a just-spun-up L4, pipeline runs there, async copy to bucket.**

120 - **The fatal timing problem:** compute needs the **whole** file, so the L4 cannot start useful work until the upload finishes. Spinning the L4 up **to receive** the upload means the GPU is **allocated and idle** for the entire 175 s upload ? the \$4.2 idle tax, same as A. Upload and compute **cannot overlap**.

121 - **Durability:** introduces an **exposure window** (upload-complete ? async-mirror-confirmed) where the only copy is on L4 local disk; a preemption/crash in that window **loses the source** unless the client re-uploads (see §4.5).

122 - **Verdict: reject.** It pays the idle-GPU tax **and** takes on a durability window, to save a transfer that is ~free and seconds-fast.

123

124 **Option A? (recommended) ? co-located Jobs, bucket-mediated upload (the L1?L3 design).**

125 - Keep **upload ? bucket** (cheap, bounded-memory, durable-immediately). Keep the **scale-to-zero L4 Job**. Move **validate + stitch** onto the Job via `phase_placement` (#521)`. Make the CP **stop downloading** (delegate validation + trust the md5, win #2). The Job pulls the file **once** from the same-region bucket at LAN speed.

126 - **Data movement:** **1 client upload + 1 intra-region LAN pull** ? the floor. (Detection and validation/stitch all run in **one** Job execution, so it's one pull, not two.)

127 - **Cost:** **+\$0.04/run GPU vs today, ~16 min CP CPU/run** (the actual scaling win). No idle tax, no warm bill, no VM.

128 - **Durability:** **no exposure window** ? the bucket copy exists before the Job starts.

129 - **Ops/parity:** **honors D4/D7/D16**; reuses the shipped dispatch/poll path; every phase is a **config flip with an in-process fallback** (#521's `_dispatch*_remote`` already logs-and-falls-back to in-process when the box isn't cloud-wired).

130

131 > **The one-line intuition:** the bucket **is** the co-located reliable disk; the GPU should attach only for the ~5 min of compute, pulling from it at LAN speed ? never sit idle for the ~3 min upload.

132

133 **### 4.4 The minimal control-plane surface (the "thin router")**

134

135 After A?, the control-plane does **only** I/O-light, decode-free work ? trivially horizontally scalable (the one caveat is SQLite; a Postgres move is out of scope, noted in §6):

136

137 | Keep on the control-plane | Why it must stay | Code anchor `[verified]`` |

138 |---|---|---|

139 | **Cloudflare-Access edge auth** (verify `Cf-Access` JWT`) | D9 ? one identity system; direct-hit spoofing guard | `backend/api/auth.py``, gated at `api/process`` |

140 | **Upload endpoint** (streams parts ? bucket) | Bounded-memory, no decode (#480); the durable-copy producer | `backend/api/uploads.py`` |

141 | **Dispatch / routing** (submit job, dispatch Cloud Run Job, bucket-poll) | Owns the async job lifecycle (`JobWaiting`/claim_due_job``) | `backend/orchestration.py``, `backend/compute/cloud_run.py`` |

142 | **DB writes** (jobs, videos, `play_segments``, validation verdict) | The control-plane owns the SQLite DB (worker uses a throwaway scratch DB) | `backend/infrastructure/database*.py`` |

143 | **SPA serving + job status** (`/api/jobs/{id}`` polling) | The UX surface | `backend/api/jobs.py``, `backend/ui/`` |

```

144 | **Signed-URL download** (`/api/highlights` 307) | Reel is already in the bucket; CP
just signs | `backend/main.py::get_highlights` |
145 | **~~Validation~~** | ? **L4** via `phase_placement.validation` (#521) | delegated |
146 | **~~Stitch/compile~~** | ? **L4** via `phase_placement.compile` (#521) | delegated |
147 | **~~Full-file download + re-hash~~** | ? **eliminated** (delegate validation + trust
md5, win #2) | this PR |
148
149 ### 4.5 The async copy-to-bucket durability model (spec ? needed *only* if a future
upload-onto-L4 path is ever adopted)
150
151 The recommended A? design **has no async-copy step** (the bucket copy is the *upload*,
produced before compute). This section specs the model the epic asked for, so that if the owner
overrides §8 and adopts upload-onto-L4, the durability semantics are pinned rather than
improvised:
152
153 - **When the mirror runs:** kick a background `storage.put(local ? gs:///uploads/?)`
**immediately on upload-complete** (not "during segmentation" ? starting it later only lengthens
the exposure window; there's no compute reason to delay it). Run it concurrently with
validation/detect.
154 - **Failure/resume:** retry with capped exponential backoff; the mirror is
**idempotent** (same object key = the md5). On worker preemption before the mirror confirms, the
run is **lost** ? the job must fail with a *retryable* status so the client can re-submit (the
source is still on the client). **Never** report success until the mirror is **confirmed**
(object exists + size/md5 match).
155 - **Exposure window:** upload-complete ? mirror-confirmed, the **only** durable copy is
L4 local PD. Window length ? mirror wall (1.83 GiB intra-region ? seconds?tens of seconds
`[design]`). A preemption in-window = re-upload. **This window is the entire reason A? is
preferred** ? A?'s window is *zero*.
156 - **Ack contract:** the client keeps the source until the API returns
`mirror_confirmed:true`; only then may it release it. (A? needs no such contract ? bucket-
first.)
157
158 ## 5. Phased migration ? every step a config flip with a bucket-mediated fallback
159
160 Built on #521's `resolve_phase_placement(cfg, phase)` so **placement is config, not
code**. Each phase is independently reversible (`control_plane` ? `cloud_run_l4`) and falls back
to in-process on a non-cloud box.
161
162 | Phase | Change | Mechanism | Fallback | Status |
163 |---|---|---|---|---|
164 | **P0** | *Today* ? detect on L4; validate+stitch on CP | `compute_target=cloud_run` |
in-process | shipped `[verified]` |
165 | **P1** | **Stitch ? L4** | `phase_placement.compile=cloud_run_l4` (+
`cloud_stitch_encoder`) | in-process stitch | **#521 (open)** |
166 | **P2a** | **Drop the CP re-hash** (trust submit-time md5) |
`deployment.trust_submit_video_id` (default on) | re-hash on the drain | **this PR (win #2)** |
167 | **P2b** | **Validation ? L4** (CP stops running validators) |
`phase_placement.validation=cloud_run_l4` (guards: requires cloud segment) | CP validates |
**#521 (open)** |
168 | **P2 net** | **CP never downloads the file** ? the ~30 s gap dies; movement 3x?2x |
P2a ? P2b together | either half alone degrades gracefully | needs #521 **** win #2 merged |
169 | **P3** | Worker serializes its **validation verdict** back (per-validator results +
#518 cache warm), so a worker-side reject renders the same SPA detail as a CP reject | new
`report.json`/marker field + CP ingest | worker reject = rc?0 + message (today) | **follow-up**
(see §6) |
170 | **P4** | *If and only if* sustained load crosses the §4.2 breakeven | persistent
worker (VM/Service) | scale-to-zero Job | **owner-gated, new ADR** |
171
172 **The "collapse onto the L4" the epic asks for = P1 + P2, both config flips on #521's
knob + this PR's win #2.** No rearchitecture, no new infra, fully reversible.
173
174 ## 6. Honest limits ? what this does NOT do / does NOT prove
175
176 - **A green suite does not prove the cloud path** ? the tests are fully-mocked/offline
(a fake `CloudRunJobClient`). P2's "CP never downloads" and the idle-GPU/round-trip numbers are

```

validated ****in the cost model + code paths****, not on live infra. A live CUJ on rev ? `00020` (owner-side, D17 budget) is required to ***confirm*** the gap is gone and the movement is 2x.

177 - ****Network-egress "free intra-region" is [researched]**, not live-verified** ? general web egress is blocked here, so same-region GCS?Cloud Run being un-billed-as-internet-egress rests on documented GCP behavior, not a fetched invoice. If it were ***not*** free, A? is ***still*** cheapest (it moves the fewest bytes); the conclusion is robust to this uncertainty.

178 - ****Win #2 changes nothing when no trusted id is present**** ? local/appliance uploads and non-GCS direct-URI submits (`submit\_time\_video\_id` ? `None`) still re-hash. The win applies to the GCS cloud-serving path (the CUJ case).

179 - ****P3 (worker validation-verdict serialization) is a real gap #521 leaves open**** `[verified]`: today a worker-side validation reject surfaces as `rc?0` + a message, but the per-validator `ValidationResult` list (`result['results']` the SPA renders) and the #518 `validation\_cache` warm are ****not**** reconstructed on the CP. Until P3, moving validation to the L4 (P2b) trades richer verdict UX + cache-warming for the CPU offload. Called out so P2b isn't flipped on blind.

180 - ****SQLite is the residual CP scaling limit**** ? the thin router is stateless ***except*** the SQLite DB; true horizontal scale-out needs Postgres (out of scope; noted).

181 - ****The #513 asymmetry reverses under P2b**** `[verified]`: worker-side validators run at model-default thresholds unless the CP forwards `validation.\*` as `--set`. #521's validation-delegation path must forward those (or re-inherit the divergence that caused the #513 incident). Flagged for the #521 review, not owned here.

182

183 **## 7. Deliverables & progress tracker ? source of truth**

184

185 Legend: ? Todo · ? In progress · ? Done · ? Gated. One small PR per row, branched from `origin/master`, no stacking.

186

	ID	Deliverable	Depends on	Gated?	Status	PR
	P0	**This design doc** (resolves topology A/B/C; recommends A?; specs durability + phased migration)	? owner-review	? (this PR)	#524	
	P1	**Win #2** ? drop the CP re-hash; trust submit-time md5; `deployment.trust_submit_video_id` (default on), reversible	? No	? (this PR)	#524	
	P2	**Win #1** ? validation-on-L4 (`phase_placement.validation`)	#521	No	?	**delivered by #521** ? not duplicated here
	P3	**CP-download-avoidance measured live** ? confirm the ~30 s gap is gone + movement is 2x on rev ? `00020`	#521 + P1 merged	? owner-side (live spend, D17)	? ?	
	P4	**Worker validation-verdict serialization** (\$6 gap) so P2b keeps rich SPA verdict + #518 cache	#521	No	? follow-up	?
	P5	**Persistent-GPU re-evaluation** only if load crosses \$4.2 breakeven	volume	?	new ADR	? ?

195

196 **## 8. Open questions ? owner / external**

197

198 ****Blocking (owner sign-off):****

199 1. ****Do you accept rejecting "upload directly onto the L4" (L5)?**** The epic proposed it; this doc recommends against it with the \$4.2/\$4.3 idle-GPU-vs-free-LAN-pull numbers. A? delivers the same "collapse onto the L4" outcome, cheaper and more durable, via config. ***If you still want upload-onto-L4*** (e.g. for a future no-bucket edge deployment), \$4.5 specs its durability model ? but it needs a Service-with-GPU or VM (a D16(a)/D7 reversal ? new ADR).

200 2. ****Flip P2b (validation?L4) before or after P3 (verdict serialization)?**** P2b alone offloads CPU but degrades the reject-UX + skips #518 cache-warm (\$6). Recommend: land P3 first, then P2b.

201

202 ****Nice-to-know:****

203 3. Is pilot volume expected to cross the \$4.2 breakeven within the planning horizon? (Decides whether P5 is ever live.)

204

205 **## 9. Definition of done**

206

207 - [x] Design doc resolves the topology question (A/B/C scored) with a clear recommendation (A?) + quantified cost/latency/reliability. `[this PR]`

208 - [x] Minimal control-plane surface defined (\$4.4). `[this PR]`

209 - [x] Async copy-to-bucket durability model specified with failure/resume + exposure window (\$4.5). `[this PR]`

```

210     - [x] Phased migration laid out on #521's `phase_placement`, each step a config flip
with a bucket-mediated fallback (§5). `[this PR]`
211     - [x] Win #2 landed (config-guarded, reversible, tests green). `[this PR]`
212     - [ ] Win #1 confirmed delivered by #521 (not duplicated). `[cross-PR]`
213     - [ ] Live CUJ confirms the gap is gone + movement is 2× (P3, owner-side). `[gated]`
214     - [ ] Owner signs off on L5 (reject upload-onto-L4) ? flip Status ? IN PROGRESS/DONE.
215
216     ## 10. References
217
218     **Internal code anchors `[verified]`:** `backend/api/uploads.py` (streamed md5),
`backend/orchestration.py:220-239` (`submit_time_video_id`, #484) / `:741-763` (re-hash) /
`_maybe_dispatch_remote`, `backend/api/jobs.py:80-153` (submit-time cache already trusts the
md5), `backend/api/job_worker.py:122-193` (`_job_to_request`/`_run_claimed_job`),
`backend/compute/cloud_run.py` (dispatch + bucket-poll), `backend/config/placement.py` (#521
`resolve_phase_placement`), `backend/worker/__main__.py` (worker validation + `--skip-
validation`).
219     **Internal docs:** `[PHASE_PLACEMENT.md` (#521)](https://github.com/Khelsutra/badminton-highlight-indexer/blob/feat/521-compile-on-l4/docs/COMPUTE\_DECOUPLED\_SERVING/PHASE\_PLACEMENT.md), [README.md](./README.md) (ADR-014,
D4/D7/D16), `runlogs/DEPLOY_REPORT_cloud-serving_validation-latency_2026-07-07.md` §9
(evidence).
220     **External `[verified]` rates:** GCP Cloud Billing Catalog API
(`cloudbilling.googleapis.com`, asia-southeast1, USD, 2026-07-08). **`[researched]`** same-
region GCS?Cloud Run egress treatment (not live-fetchable here).
221
222     ---
223
224     ### Changelog
225     - `2026-07-08` ? initial draft: topology A/B/C resolved (recommend A? = co-located Jobs
+ bucket upload; reject persistent-GPU + upload-onto-L4 with a cost model); durability + phased
migration on #521's `phase_placement`; win #2 landed alongside.
226

```

7. assistant (2026-07-08T06:27:54.531Z)

Now let me verify the code anchors cited in the doc. Let me read the relevant files.

8. user (2026-07-08T06:27:56.099Z)

```

215         if minutes < 1:
216             return f"segmenting ({seg_name}) on cloud GPU ? starting up"
217         return f"segmenting ({seg_name}) on cloud GPU ? {minutes} min elapsed"
218
219
220     def submit_time_video_id(input_ref, cfg) -> Optional[str]:
221         """The video's md5 (the canonical ``video_id``) resolvable CHEAPLY at submit time ?
gs://
222         object metadata only, one stat, no bytes (#484). ``None`` ? unknown: local files
keep
223         deferred hashing on the drain (DEFERRED_HARDENING #16), and composite objects carry
no
224         ``md5Hash``. Never raises ? a metadata miss must not block a submit."""
225         if getattr(input_ref, "backend", "") != "gcs":
226             return None
227         sc = _storage_settings(cfg)
228         if not sc:
229             return None
230         try:
231             st = storage.get_storage("gcs", config=sc).stat(input_ref)
232             return getattr(st, "md5", None) or None
233         except Exception: # noqa: BLE001 ? best-effort identity, the drain still hashes
234             logger.info(
235                 "[API] submit-time md5 unavailable for %s (metadata stat failed)",
236                 getattr(input_ref, "uri", input_ref),
237                 exc_info=True,

```

```

238         )
239         return None
240
241
242     def _looks_like_video_id(value: Optional[str]) -> bool:
243         """True iff ``value`` is a well-formed ``video_id`` ? the 32-char lowercase hex MD5
that
244         ``compute_video_id`` produces (``backend/utils/hashing.py``). Guards the #524 win-#2
trust path:
245         a trusted id is only reused in place of a re-hash when it has the EXACT shape the
drain would
246         have computed, so a malformed / truncated / wrong-encoding id can never become the
DB key ?
247         it degrades to a real hash instead. ``submit_time_video_id`` already normalizes the
gs://
248         base64 md5 to this hex form (#484, ``backend/storage/gcs.py::_md5_hex``)."""
249         return (
250             isinstance(value, str)
251             and len(value) == 32
252             and all(c in "0123456789abcdef" for c in value)
253         )
254
255
256     def _cached_process_payload(
257         video_id: str, validation_results, segments, requested_compute
258     ) -> Dict[str, Any]:
259         """The /api/process result for a video whose detection already exists ? ONE shape
for

```

9. user (2026-07-08T06:27:57.067Z)

```

755         config = None
756         # Resolve config/db from module globals when not passed explicitly
757         if config is None:
758             config = _main.global_config
759         if db is None:
760             db = _main.db_instance
761
762         notify = progress or (lambda stage: None)
763
764         notify("hashing")
765         # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
(identity ?
766         # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
original
767         # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
768         # consumers (hash / validators / segmenter) use the resolved local path.
769         local_video = ""
770         try:
771             local_video = storage.acquire_local_input(
772                 req.video_path, getattr(config, "storage", None)
773             )
774             # #524 win #2: trust the SERVER-DERIVED submit-time md5 instead of re-hashing
the whole
775             # (often multi-GB) clip we just fetched. The trusted id is byte-identical to
776             # compute_video_id (same bytes, same MD5 ? #484) and is ALREADY the value the
submit-time
777             # cache probe trusts, so reusing it here is consistent, not a new trust
assumption. Gated by
778             # deployment.trust_submit_video_id (default on) and only honoured when the id is
well-formed
779             # (_looks_like_video_id) ? any absence/malformation degrades to a real hash. The
fetch above
780             # is unchanged; only the redundant hash is skipped.

```

```

781         trust_submitted = bool(
782             getattr(
783                 getattr(config, "deployment", None), "trust_submit_video_id", True
784             )
785         )
786         # STRICT: trust only an id in the EXACT form the drain would have produced
(32-char
787         # lowercase hex, per compute_video_id / submit_time_video_id). No normalization
? a
788         # differently-cased or malformed id is a signal something is off, so re-hash
rather than
789         # silently coerce it into the DB key.
790         candidate = trusted_video_id or ""
791         if trust_submitted and _looks_like_video_id(candidate):
792             video_id = candidate
793             logger.info(
794                 "[API] trusting submit-time video_id=%s ? skipped the drain re-hash
(#524).",
795                 video_id,
796             )
797         else:
798             video_id = _main.compute_video_id(local_video)
799     except Exception:

```

10. user (2026-07-08T06:28:06.277Z)

```

16:``ProcessRequest``, ``compute_video_id``, ``emit_indexer_report``, ``VideoCompiler``.
220:def submit_time_video_id(input_ref, cfg) -> Optional[str]:
708:def _execute_processing(
748:    local_video = storage.acquire_local_input(
751:        video_id = _main.compute_video_id(local_video)
1159:    local_src = storage.acquire_local_input(

```

11. user (2026-07-08T06:28:27.305Z)

```

=== gcs.py _md5_hex ===
24:def _md5_hex(md5_b64: "str | None") -> Optional[str]:
25:    """GCS reports ``md5Hash`` as base64 of the raw digest; ``compute_video_id`` (the DB key)
27:    have no md5Hash ? None; a malformed value is treated as absent, never raised."""
28:    if not md5_b64:
31:        raw = base64.b64decode(md5_b64, validate=True)
83:        md5=_md5_hex(getattr(b, "md5_hash", None)),
=== uploads.py md5 ===
15: ``input_root``) plus the streamed md5 as ``video_id``.
27:import hashlib
185:    "hasher": hashlib.md5(),
300:    # Remote staging: close() finalizes the streamed object; the md5 accumulated per
316:    "video_id": st["hasher"].hexdigest(),

```

12. user (2026-07-08T06:28:27.869Z)

```

=== jobs.py 80-153 ===
def submit_process_job(
    db: Any, cfg: Any, job_id: str, req: Any, owner_id: Optional[str], input_ref: Any
) -> Optional[Tuple[bool, JSONResponse]]:
    """The whole /api/process submit decision (#448 dedup + #484 submit-time cache) in one hook.

    Order matters (#492 review):
    1. Resolve a cheap md5 (gs:// metadata) and PROBE the cache ? read-only, no side effects.
    2. The remote-input scratch guard (507) runs ONLY on a probe miss: a cache hit never
       fetches to local scratch, so low temp space must not reject it (review blocker 2).
    3. CLAIM atomically (#481). A concurrent identical submit dedups here ? BEFORE any
       re-hydrate side effect, so the first request owns the cache completion (blocker 1).
    4. Only the claim WINNER realizes the hit (``cached_process_result`` ? itself serialized

```

per video_id) and completes the job instantly: no drain, no GPU dispatch.

```
Returns ``queued, response``: ``queued`` True ? the caller kicks the drain; ``None`` ?
DB failure (the caller 500s)."""
from backend import orchestration # call-time seam: tests patch orchestration attrs

video_id = orchestration.submit_time_video_id(input_ref, cfg) or ""
probed = bool(video_id) and not req.force and orchestration.probe_process_cache(
    db, cfg, video_id
)
if input_ref.backend != "local" and not probed:
    _fetch_scratch_guard(cfg, input_ref) # raises 507 pre-claim, exactly as before
outcome = claim_process_job(db, job_id, req, owner_id, video_id=video_id)
if outcome is None:
    return None
created, dedup_response = outcome
if not created:
    assert dedup_response is not None # claim contract: not-created ? dedup response
    return (False, dedup_response)
if probed:
    cached = orchestration.cached_process_result(
        db, cfg, video_id, req.video_path, requested_compute=req.compute
    )
    if cached is not None:
        db.mark_job_done(job_id, cached)
        return (
            False,
            JSONResponse(
                status_code=202,
                content={
                    "job_id": job_id,
                    "status": "done",
                    "poll": f"/api/jobs/{job_id}",
                    "cached": True,
                },
            ),
        )
    # The probe said hit but realizing it failed (report vanished / corrupt csv): fall
    # back to the normal queued dispatch ? but the scratch guard was skipped above, so it
    # must run NOW. If it fails, the already-claimed row is marked failed BEFORE raising
    # (a bare raise would leave a phantom 'queued' job for the drain to grab).
    if input_ref.backend != "local":
        try:
            _fetch_scratch_guard(cfg, input_ref)
        except HTTPException:
            db.mark_job_failed(
                job_id,
                "cache re-hydrate failed and local scratch cannot hold a fetch of "
                "the source; free disk space or retry later",
            )
            raise
    return (
        True,
        JSONResponse(
            status_code=202,
            content={
                "job_id": job_id,
                "status": "queued",
                "poll": f"/api/jobs/{job_id}",
            },
        ),
    )
)
```

```

24     def _md5_hex(md5_b64: "str | None") -> Optional[str]:
25         """GCS reports ``md5Hash`` as base64 of the raw digest; ``compute_video_id`` (the DB
key)
26         is a HEX digest ? convert so the two are directly comparable (#484). Composite
objects
27         have no md5Hash ? None; a malformed value is treated as absent, never raised."""
28         if not md5_b64:
29             return None
30         try:
31             raw = base64.b64decode(md5_b64, validate=True)
32         except (binascii.Error, ValueError):
33             return None
34         return raw.hex() if len(raw) == 16 else None
35
36
37     class GCSStorage(StorageBackend):
38         def __init__(
39             self,
40             *,
41             client: Any = None,
42             project: Optional[str] = None,
43             signed_url_ttl: int = 3600,

```

14. user (2026-07-08T06:28:44.498Z)

```

=== placement.py on #521 branch ===
"""Phase ? machine placement resolution (#521).

```

A **phase placement** answers one question per pipeline phase ? ***where does the heavy compute for this phase run?*** ? with a config knob (`deployment.phase_placement`) instead of code. The three placeable phases and their values:

```

validation ? control_plane | cloud_run_l4
segment     ? control_plane | cloud_run_l4 (a.k.a. "detect")
compile     ? control_plane | cloud_run_l4 (a.k.a. "stitch")

```

`control_plane` runs the phase IN-PROCESS on the (CPU-only) control-plane service;
`cloud_run_l4` dispatches it to the GPU/NVENC Cloud Run worker Job (the same L4 detection already uses). This module is the SINGLE source of truth so `backend.orchestration` (segment + compile) and the worker never re-derive the mapping.

Default = today's behaviour (pure / additive). When a phase's knob is unset (`None`) the resolver reproduces the pre-#521 placement:

```

validation ? control_plane (the authoritative gate,
#512/#513)
segment     ? cloud_run_l4 IFF deployment.compute_target == "cloud_run", else control_plane
compile     ? control_plane (the in-process ffmpeg
stitch)

```

so a config with no `phase_placement` block behaves byte-identically to before this module existed.

```

from __future__ import annotations

```

```

from typing import Any

```

Phase identities (kept as constants so callers don't stringly-type the phase names).

```

PHASE_VALIDATION = "validation"

```

```

PHASE_SEGMENT = "segment"
PHASE_COMPILE = "compile"
PHASES: tuple[str, ...] = (PHASE_VALIDATION, PHASE_SEGMENT, PHASE_COMPILE)

```

Placement targets.

```

PLACE_CONTROL_PLANE = "control_plane"
PLACE_CLOUD_L4 = "cloud_run_l4"
PLACEMENTS: tuple[str, ...] = (PLACE_CONTROL_PLANE, PLACE_CLOUD_L4)

```

```

def resolve_phase_placement(cfg: Any, phase: str) -> str:
    """Where does ``phase`` run ? ``"control_plane"`` or ``"cloud_run_l4"``?

```

Precedence: an explicit ``deployment.phase\_placement.<phase>`` wins; otherwise the legacy default

(mirrors the pre-#521 behaviour, see the module docstring). Never raises ? an absent ``deployment`` / ``phase\_placement`` degrades to the legacy default.

```

"""

```

```

deployment = getattr(cfg, "deployment", None)
placement = getattr(deployment, "phase_placement", None)
explicit = getattr(placement, phase, None) if placement is not None else None
if explicit:

```

```

    return str(explicit)

```

Legacy defaults ? reproduce today's placement exactly.

```

if phase == PHASE_SEGMENT:

```

```

    compute_target = (getattr(deployment, "compute_target", "") or "").strip()

```

```

    return PLACE_CLOUD_L4 if compute_target == "cloud_run" else PLACE_CONTROL_PLANE

```

validation + compile default to the control-plane (their pre-#521 home).

```

return PLACE_CONTROL_PLANE

```

15. user (2026-07-08T06:28:56.903Z)

33:| D4 | In **cloud-serving**, STITCH runs on Cloud Run** (co-located with detect) so CPU/wall + egress are metered into the per-job cost ledger. | owner (a) | Stitch-on-laptop would hide real cost. |

36:| D7 | **Cloud Run cold-start: SCALE-TO-ZERO** ? accept the ~1?3 min cold-start (it's an async job: upload?poll?reel, amortized over the multi-minute job). **No warm pool** (that ~\$500/mo idle would break per-video billing). | owner 2026-06-21 | M6/M7. Revisit a warm lane only on a UX complaint. |

45:| D16 | **M6/M7 implementation SHAPE** + the cheap \$8 picks (owner 2026-06-21):** (a) **Cloud Run JOBS** (not Services-with-GPU) for the detect+stitch unit, **bucket-poll** for completion** (reuses `JobWaiting`/`claim\_due\_job` verbatim, fits async upload?poll?reel + scale-to-zero D7); (b) **stitch ENCODE = libx264** (protects the D5 byte-determinism/parity guarantee + the L4 parity test) ? **NVDEC** for DECODE only** (a speed lever that never touches output bytes); **NVENC** encode is DEFERRED behind a config knob**, revisit only if M7 cost data justifies relaxing parity (\$8 Q8); (c) **input cap = hard-reject <2GB**** for v1 (a config cap so raising to 4GB later is one line; matches the shipped M2 chunked-upload threshold ? \$8 Q9). | owner 2026-06-21 | M6/M7. Resolves \$8 Q2 (co-located, = D4), Q8, Q9, Q10 (dir name confirmed). |

16. user (2026-07-08T06:28:58.931Z)

14:> **#524 (2026-07-08):** the **L4-primary topology** question ? "collapse upload?stitch onto the worker, shrink the control-plane to a thin router" ? is resolved in [L4_PRIMARY_TOPOLOGY.md](./L4_PRIMARY_TOPOLOGY.md). It **recommends against** uploading directly onto the GPU box (a cost model shows that trades a ~free intra-region LAN pull for minutes of idle GPU) and instead **collapses validate+detect+stitch** onto the scale-to-zero L4 Job** via #521's `phase\_placement` ? honoring D4/D7/D16. Ships **win #2** here (drop the redundant control-plane re-hash; trust the submit-time md5, reversible).

17:The rally engine is a **pure function** of `(input bytes + config) ? outputs` in three deterministic stages ? **DETECT** ? **WINDOW** ? **STITCH** ? that **no deployment profile** may branch on**. A **ComputeBackend** (deployment profile)** decides *where* the core runs (a local process vs a Cloud Run GPU job); a **StorageBackend**** decides *where bytes come from* (local FS / USB / mounted Google Drive / bucket). **One startup config** picks both. We ship **truly-local** (local GPU + in-process stitch, zero cloud) and **cloud-serving** (upload <2GB / gdrive /

bucket ? a Cloud Run GPU job runs detect ****and**** stitch, so stitch's compute is metered), plus ****cpu-mock**** for CI. The single most important caveat: detection is ****byte-identical only on the same GPU**** (cross-GPU is sub-pixel-different, per the 2026-06-20 deploy report) ? so the ****parity guarantee is enforced at the WINDOW + STITCH layer****, not detector-CSV bytes.

20:- ****Why now:**** DECOUPLED_COMPUTE proved compute ***portability*** (M0?M2 + M3.5 on a GCP L4) and is archived; its deferred ****M3** (serving endpoint + per-video billing) + M4 (scale-to-zero + cost guard)****** become this project. The owner needs ****both**** a hosted service ***and*** a truly-local mode, with the ****core (detect+stitch) unaffected**** by which one runs.

22: - **** (a) Hosted service**** ? operate on small ****local uploads (<2GB)**** ***and*** load from ****gdrive / a bucket****, spinning up ****Cloud Run for both rally segmentation AND stitching**** (stitching is compute-intensive ? run it where its cost is accounted for).

33:| D4 | In ****cloud-serving, STITCH runs on Cloud Run**** (co-located with detect) so CPU/wall + egress are metered into the per-job cost ledger. | owner (a) | Stitch-on-laptop would hide real cost. |

36:| D7 | ****Cloud Run cold-start: SCALE-TO-ZERO**** ? accept the ~1?3 min cold-start (it's an async job: upload?poll?reel, amortized over the multi-minute job). ****No warm pool**** (that ~\$500/mo idle would break per-video billing). | owner 2026-06-21 | M6/M7. Revisit a warm lane only on a UX complaint. |

45:| D16 | ****M6/M7 implementation SHAPE + the cheap \$8 picks (owner 2026-06-21):**** (a) ****Cloud Run JOBS**** (not Services-with-GPU) for the detect+stitch unit, ****bucket-poll for completion**** (reuses `JobWaiting`/`claim\_due\_job` verbatim, fits async upload?poll?reel + scale-to-zero D7); (b) ****stitch ENCODE = libx264**** (protects the D5 byte-determinism/parity guarantee + the L4 parity test) ? ****NVDEC for DECODE only**** (a speed lever that never touches output bytes); ****NVENC encode is DEFERRED behind a config knob****, revisit only if M7 cost data justifies relaxing parity (\$8 Q8); (c) ****input cap = hard-reject <2GB**** for v1 (a config cap so raising to 4GB later is one line; matches the shipped M2 chunked-upload threshold ? \$8 Q9). | owner 2026-06-21 | M6/M7. Resolves \$8 Q2 (co-located, = D4), Q8, Q9, Q10 (dir name confirmed). |

58:| ****ADR-012**** | GPU-native; TPU deferred; ****Cloud Run+GPU scale-to-zero = serving model****; ****NVDEC**** named as the decode lever | The serving substrate + the 7-step seed scope this expands. |

72:3. ****STITCH (CPU/ffmpeg):**** `VideoCompiler.compile\_highlights(raw\_video, segments, out\_name) ? reel` (`stitcher/compiler.py:303`). Pure FS+subprocess (merge ? ffmpeg ? watermark ? per-clip libx264 trim ? concat). Deterministic for a given input + ranges + watermark config.

80:| ****cloud-serving**** | `compute\_target=cloud\_run`; `detector\_impl=native` (L4, cuda); ****detect AND stitch on the same Cloud Run job**** | `gcs` (or gdrive mount / assembled upload); per-video workspace; reels via signed URL | Cloud Run control plane enqueues ? Cloud Run GPU job pulls/runs/pushes; `WAIT\_AND\_RESUME` reschedules the control-plane job; scale-to-zero | Hosted SaaS ? owner (a) |

92:### 4.4 Compute placement ? and ***why stitch runs on Cloud Run*** `[design]`

96:****Exists:**** the 3-stage core; `\_resolve\_runner` (the single compute seam); `StubDetectorRunner` (`replay\_csv` = byte-exact \$0 anchor); `StorageBackend` ABC + 4 backends + `StorageRef.parse\_uri`; the worker CLI (fetch/put, `--config/--set`); the async job control plane (`jobs` table, `claim\_due\_job` CAS); `ExecutionPolicy WAIT\_AND\_RESUME`/`JobWaiting` (carries to Cloud Run dispatch verbatim); the chunked-upload router (<2GB); `skip\_ai\_handoff` (LocalNoAI); storage fakes; golden fixtures + experiment harness. ****Partial:**** per-video workspace helper (default-OFF); `DeviceContext` (designed); edge-auth (v4 `user\_id` columns exist). ****Net-new:**** the `deployment` section; `input\_backend`; configurable output base (kill hardcoded `output/`); Cloud Run dispatch shim + container image; cost ledger + pricebook; edge-auth header extraction; startup env detection.

124:| M7 | ****cloud-serving profile**** e2e on L4: upload<2GB + gdrive/bucket ? detect+stitch on Cloud Run ? reel via signed URL; ****DEPLOY_REPORT**** (posterity, sample runs + contact sheet) | M6 | ? owner (spend) | ? | ? |

134:****?** The 5 gating questions are LOCKED (owner, 2026-06-21) ? see \$2 D7?D12:******

135:1. ? ****Cold-start ? D7**** (scale-to-zero, accept cold-start).

141:****?** The remaining smaller picks are LOCKED (owner, 2026-06-21) ? see \$2 D16:******

142:2. ? ****Stitch placement ? co-located**** in the detect job (= D4; one metered unit; the job table supports a future CPU-split non-breaking, revisit only on bursty-compile metrics).

144:8. ? ****Codec ? D16:**** ****libx264 ENCODE**** (protect parity) + ****NVDEC DECODE-only**** (speed, no output-byte change); NVENC-encode deferred behind a config knob.

145:9. ? ****Input cap ? D16:**** ****hard-reject <2GB**** for v1, as a config cap (raise to 4GB later in one line).

166:- 2026-06-21 ? ****M5+ batch authorized + design picks locked**** (owner): the remaining \$8 picks resolved ? ****D16**** (M6 = Cloud Run ****Jobs**** + bucket-poll; ****libx264 encode + NVDEC decode-only****; ****hard-reject <2GB**** input cap; dir name confirmed) and ****D17**** (real cloud verification budget = ****\$100/?10k cap****, delete-after). Owner gave a blanket go to write+merge

the ****default-OFF code**** for every remaining gated milestone (M5,M6,M8,M9-auth,M9-consent-cols,M11-plumbing,M12), one isolated PR each, reserving the gate for the real run/spend/calibration/counsel. Grounded by gate-map workflow `\wf_4849400f\`.
170:- 2026-06-21 ? ****M6 code landed**** ([#287](https://github.com/Khelsutra/badminton-highlight-indexer/pull/287)): `\backend\compute\` ComputeBackend registry ? \CloudRunCompute\` dispatches a Cloud Run **Job** (injectable client) then **bucket-polls** a done-marker via \execution_policy.poll_until\` + the storage layer (D16); \get_compute_backend\` returns a backend ONLY for \compute_target=cloud_run\` (else \None\` ? the worker's in-process path, **default-OFF**). The dispatched container runs **inline** (forced \local_gpu\`+native, never re-dispatch); the worker writes a \--done-uri\` completion marker on success.\deploy\cloudrun\{Dockerfile,README}\` = the L4 image (CUDA+ffmpeg+fonts+\wasb\` conda env; weights staged per WASB-C3) + the M7 runbook. Adversarial review (\wf_87b22fa4\`) folded a **blocker** (cloud \fetch\` needs a dst ? reads silently returned \{\}\` on gcs/s3) + a poll-wait test gap + \PolicyTimeout\`?clean rc 4. Suite green, ruff+mypy clean. **? M6 ? ? the real build\push\L4 run + DEPLOY_REPORT is M7 (owner, D17 budget).**`

17. user (2026-07-08T06:30:01.135Z)

```
ls: cannot access 'runlogs/': No such file or directory
=== search numbers in runlog ===
file=
```

18. user (2026-07-08T06:30:02.897Z)

```
=== worker --skip-validation / thresholds ===
4:      [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
10:``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini key) ?
49:      """"Coerce a ``--set`` string value to bool/int/float where it round-trips, else str.""""
66:          raise ValueError(f"--set expects k=v, got {kv!r}")
194:      skip_validation=bool(getattr(args, "skip_validation", False)),
277:      # resolves different thresholds than the control-plane's ? observed 2026-07-07: a
5312x2988 clip
278:      # (sharpness 11.9) PASSED the control-plane at validation.min_blur_threshold=8.0, then
the worker
280:      # --skip-validation (set by the dispatcher in orchestration._maybe_dispatch_remote)
skips it; the
282:      if getattr(args, "skip_validation", False):
284:          "[worker] validation SKIPPED (--skip-validation): the dispatcher is the
authoritative "
747:          "--set",
751:          help="config override, e.g. --set indexing.skip_ai_handoff=true",
754:          "--skip-validation",
809:      pt.add_argument("--set", action="append", default=[], metavar="k=v")
```

19. user (2026-07-08T06:30:16.381Z)

```
=== find runlogs ===
=== git ls-files runlogs ===
docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_2026-07-04.md
docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_a1-cuj_2026-07-04.md
docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_a1-e2e_2026-07-04.md
docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_alpha-all1v1_2026-07-06.md
docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_cp-rebuild_2026-07-05.md
docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_gpu-resident-
nvdec_2026-07-07.md
docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_validation-
latency_2026-07-07.md
docs/COMPUTE_DECOUPLED_SERVING/runlogs/README.md
docs/COMPUTE_DECOUPLED_SERVING/runlogs/RUNLOG_cloud-serving_exec-status-fastfail_2026-07-07.md
docs/COMPUTE_DECOUPLED_SERVING/runlogs/RUNLOG_truly-local_TEMPLATE.md
=== gitignore runlogs ===
```

20. assistant (2026-07-08T06:30:26.856Z)

The runlog is at `docs/COMPUTE\_DECOUPLED\_SERVING/runlogs/`. Let me verify the cited numbers against it.

21. user (2026-07-08T06:30:31.753Z)

```
13:| **#512 speedup (off-box A/B, breadth)** | ? 1080p H.264 **1.95x**, 4K HEVC **3.10x**, 5.3K
10-bit HEVC **3.23x**; every validator decision unchanged OLD?NEW |
14:| Live control-plane "after" number | ? **live-corroborated 2026-07-08** ($9): rev `00019`
validated a fresh 1.83 GiB / 174 s clip in **183 s** (in the 2?3 min band) + reel compiled end-
to-end. Video-1's exact 412 s?~170 s A/B stays a projection (the live CUJ used a different,
heavier clip). |
22:- **Corpus:** Video 1 `gs://khehsutra-rally-
serving/videos/uploads/702636eed76145e88aa59c3000c7e415/Video 1 - Badminton.MP4` (1.41 GiB,
5312x2988 yuv420p10le HEVC, 3019 frames, MD5 `8fe6b719603dd7a7b2b6ef543139a4a3`);
`gs://khehsutra-rally-corpus/videos/golden_1080p/Boxhill_Doubles.mp4` (1080p H.264);
`gs://khehsutra-rally-corpus/videos/golden/GX010128.mp4` (2.18 GiB, 4K HEVC, 45298 frames).
57:| Control-plane validation | **411?412 s** (6m52s) ? passed (baked `min_blur=8.0`) |
**projected ~170 s (2?3 min)**; **live-corroborated 2026-07-08 ? 183 s** on rev `00019` ($9 ? a
fresh 1.83 GiB clip, not a Video-1 re-run) |
77:| GX010128 | 4K HEVC (45298 fr) | 35.76 s | 11.54 s | **3.10x** | 6/6 identical (all PASS) |
93:- **Live end-to-end CUJ is owner-gated.** ? **CLOSED 2026-07-08 ? see $9** (owner drove one
live SPA upload on rev `00019`; both ? cells now measured). Original limit, retained for
context: `POST /api/process` on rev 00018 requires a Cloudflare-Access JWT
(`edge_auth_enabled=true`; a direct call returns **401** `missing Cf-Access-Jwt-Assertion
header`). So the two ? cells ? the **live control-plane validation wall on rev 00018** and the
**stitched reel** ? need the owner to drive one upload (or hand over a CF-Access JWT / service
token). Everything else here is measured on the real serving infra. **Recipe to close them:**
upload "Video 1 - Badminton.MP4" via the SPA (compute=cloud), then `gcloud logging read
'?service_name="rally-control-plane"?("Running validations" OR "passed checks")'` for the wall,
and confirm the dispatched worker exec logs `validation SKIPPED` + emits rallies + the reel
compiles. Expect CP validation ~2?3 min and a PASS.
103: ? **CLOSED ? #512 and #513 are proven on live cloud-serving hardware.** #513 converts the
motivating soft-focus 5.3K clip from a post-dispatch FAIL into a 13-rally PASS by eliminating
the worker's redundant re-validation (169 s + wrongful blur-reject ? 0 s); #512 speeds the
frame-sampling validators **2.43x on the real L4** and **1.95?3.23x off-box across codecs**,
with **zero decision drift** OLD?NEW on every clip. The last owner-gated residual is now
**closed live 2026-07-08 ($9)**: an owner-driven SPA CUJ on rev `00019` validated a fresh 1.83
GiB / 174 s clip in **183 s** (in the projected 2?3 min band) and compiled an 11-clip reel end-
to-end ? with the worker `validation SKIPPED` on a genuine control-plane dispatch. (The
Video-1-specific 412 s?~170 s A/B stays a projection by design ? the live CUJ used a different,
heavier clip.) Validation-latency work is done.
105:## 9. Update (2026-07-08) ? live SPA CUJ closes the two owner-gated cells (on rev `00019`)
110:1. **Live rev rolled `00018` ? `00019`.** Traffic is 100% on **`rally-control-
plane-00019-96f`**, image `rally-worker@sha256:c6393d27b4ae?` (tag `:latest`), deployed
2026-07-07T17:06Z ? ~51 min after the `00018`/`8045f54e` cutover $2 documents; the worker Job is
on the same `:latest`=`c6393d27` digest (both surfaces match). It's an undocumented forward-
bump, but #512 (`20862d7`) and #513 (`23c1868`) are ancestors of `origin/master`, `f5c9dd9` (the
`00018` src) is an ancestor of master, and the run **empirically** shows both fixes live (one-
pass validation wall + worker `validation SKIPPED`). Measured on what is actually serving.
111:2. **A fresh clip, not Video 1.** The upload was **"First CUJ.MP4"*** ? **1.83 GiB**
(1,960,043,219 B), source duration **174.17 s** (?84 Mbps, 4K-class ? *heavier* than Video 1's
1.41 GiB), video_id `2f31b45e?`. So the live wall **corroborates** the Video-1 projection on a
comparably-heavy clip; it is **not** a re-measurement of Video 1's 412 s?~170 s A/B (a different
clip has a different absolute wall).
113:**Full phase breakdown (UTC 2026-07-08, rev `00019`):**
117:| 1 | Upload (client?server, 80-part multipart) | 04:25:56 ? 04:28:51 | **~2m55s** | client
uplink | 1.83 GiB @ ~84 Mbps ? owner bandwidth, not infra |
118:| 2 | Accept ? validation start (stage) | 04:28:52 ? 04:29:22 | ~30 s | control-plane |
stage upload to serving bucket |
119:| 3 | **Validation (#512 shared-decode)** ? | 04:29:22 ? 04:32:25 | **183 s (3m03s)** |
control-plane 2 vCPU | one-pass scene_cut/blur/relevance; PASSED |
120:| 4 | Dispatch ? worker ready (cold start) | 04:32:25 ? 04:33:20 | ~55 s | Cloud Run Job |
image pull + GCSFuse mount + torch/cuda init |
122:| 6 | Detection / segment (WASB) | 04:33:24 ? 04:36:20 | **161.6 s** | L4 (cuda) | 15
candidate windows, 2561 visible pts |
```

```

125:| 8 | **Compile / stitch** ? slow | 04:37:26 ? 04:50:21 | **~12m55s** | control-plane 2 vCPU
| 11 clips, ~67 s/clip 4K re-encode+watermark; concat ~2 s; mirror ~11 s |
129:**Cell 1 ? control-plane validation AFTER (live):** **183 s (3m03s)** on rev `00019`, PASSED
? the #512 shared-decode validators on the real 2-vCPU serving path, in the projected **2?3
min** band on a clip *heavier* than Video 1 (pre-#512 per-sample seeks were the ~6m52s-class
baseline). Evidence: `Running validations 04:29:22.789 ? Video passed checks 04:32:25.780`.
131:**Cell 2 ? end-to-end reel (live):** `/api/compile` stitched **11 of the 15** detected
rallies (0.5 s pad, per-clip branding watermark) from the 174.17 s source ? **`gs://khehsutra-
rally-serving/highlights/2f31b45e?/first-cuj-ks-highlight-reel.mp4`** (**581.84 MiB**, saved
04:50:10 *"Dynamic highlight reel saved (581.8 MB)"*, mirrored 04:50:21, served via
`/api/highlights` 307). **Reel ? produced end-to-end on the live control-plane** ? source fetch
? per-clip trim+watermark ? concat ? mirror-upload, all exercised.
135:**? Finding ? stitch is the scaling bottleneck (follow-ups filed).** The compile ran **~13
min on the CPU-only 2-vCPU control-plane** (~45?90 s per ~7 s clip: the watermark forces a full
4K re-encode, no NVENC). With validation (183 s) *also* on the control-plane and serialized by
`_LOCAL_COMPUTE_LANE`, one run pins the control-plane for ~16 min of heavy CPU ? it won't scale
past a few concurrent users. Filed: **#521** (move compile/stitch to the L4 worker + make
phase?machine placement config-driven) and **#520** (step-level debug tracing, server+UI, to a
low-latency sink ? this breakdown had to be hand-correlated across 3 log streams).
137:**Cost (heuristic):** one L4 detect exec (`rally-worker-64bhj`, ~222 s) ? **$0.15?0.55**
(rate-dependent, dominant) + control-plane CPU for validation+stitch (~960 s @ 2 vCPU) ?
**~$0.05** + reel egress (581 MB, if downloaded to internet) ? **~$0.07** + storage/ops **<
$0.02** + Gemini **$0** (skip_ai_handoff). **? $0.3?0.7 total**, ~one L4 exec as projected ? but
the stitch's real cost is **concurrency** (~16 min of control-plane heavy CPU), not dollars.
139:**Net:** both owner-gated cells are now live-measured on rev `00019` ? validation wall **183
s** (corroborating the 2?3 min projection) and the reel compiles end-to-end. #512 one-pass
validation and #513 skip-validation-on-real-dispatch are now both directly observed in prod. The
Video-1 412 s?~170 s A/B remains a projection by design (different, heavier clip).

```

22. assistant (2026-07-08T06:31:27.343Z)

Let me examine the upload path to understand what the ~30s "Accept?validate" gap actually is, since the runlog labels it differently than the doc.

23. user (2026-07-08T06:31:29.294Z)

```

280         "upload_id": upload_id,
281         "received_parts": st["parts"],
282         "received_bytes": st["bytes"],
283     }
284
285
286     @router.post("/api/upload/{upload_id}/complete")
287     def upload_complete(upload_id: str, req: UploadCompleteRequest):
288         """Finalize: verify part count, move into place, return {path, video_id} for
/api/process."""
289         upload_dir, _, _, _ = _upload_cfg()
290         with _uploads_lock:
291             st = _uploads.get(upload_id)
292             if st is None:
293                 raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
294             if st["parts"] == 0 or st["parts"] != req.total_parts:
295                 raise HTTPException(
296                     status_code=409,
297                     detail=f"Expected {req.total_parts} part(s), received {st['parts']}.",
298                 )
299             if st.get("writer") is not None:
300                 # Remote staging: close() finalizes the streamed object; the md5 accumulated per
301                 # part IS compute_video_id's digest (same bytes, same algorithm), so
/api/process
302                 # gets the identity without anyone re-reading the video. (Sync endpoint ?
FastAPI
303                 # runs it on the threadpool, so the blocking finalize doesn't stall the loop.)
304                 try:
305                     st["writer"].close()

```

```

306         except Exception as e: # noqa: BLE001 ? surface a clean verdict, then clean up
307             _abort_upload(upload_id)
308             raise HTTPException(
309                 status_code=502,
310                 detail=f"Failed to finalize the staged upload ({e}). Re-init to retry.",
311             ) from e
312         with _uploads_lock:
313             _uploads.pop(upload_id, None)
314         return {
315             "path": st["stage_uri"],
316             "video_id": st["hasher"].hexdigest(),
317             "size_bytes": st["bytes"],
318             "next": "POST /api/process with this path",
319         }
320         final_path = os.path.join(upload_dir, st["filename"])
321         if os.path.exists(
322             final_path
323         ): # never overwrite an earlier upload with the same name
324             stem, ext = os.path.splitext(st["filename"])

```

24. user (2026-07-08T06:31:32.033Z)

```

9:* a remote input backend with an ``input_root`` (the hosted alpha: gcs): parts STREAM
10: through a ``storage.open_writer`` into ``<input_root>/uploads/<upload_id>/<name>``
11: while an incremental MD5 accumulates ? the assembled file never touches local
14: returns the staged URI (inside the #465 input jail by construction ? it is under
15: ``input_root``) plus the streamed md5 as ``video_id``.
94: def _staging_target() -> "Optional[Dict[str, Any]]":
95:     """The remote staging root + storage settings, or None for the local-file flow.
97:     Remote staging is selected by the same input config /api/process resolves against
98:     (storage.input_backend + input_root), so a staged upload is submittable verbatim.
106:     input_root = (getattr(sc, "input_root", "") or "").rstrip("/")
107:     if input_backend == "local" or not input_root:
110:     return {"root": input_root, "config": cfg}
120:     # A streaming writer has no cancel ? close() finalizes whatever was staged, then
128:     ref = storage.parse_uri(st["stage_uri"])
169:     staging = _staging_target()
170:     if staging is not None:
171:         # Remote staging: stream parts straight into the bucket under the input root ?
172:         # never assembled on local disk/tmpfs (CONTROL_PLANE_DURABLE_REBUILD P2). The
175:         stage_uri = f"{staging['root']}/uploads/{upload_id}/{name}"
177:         writer = storage.open_writer(stage_uri, config=staging["config"])

```

25. assistant (2026-07-08T06:32:27.616Z)

Let me verify remaining anchors and run the markdown link checker as the task requests.

26. user (2026-07-08T06:32:35.595Z)

```

=== job_worker 122-193 anchors ===
1 def _job_to_request(job: Dict[str, Any]):
2     db,
3     # compute_video_id) so it can skip re-hashing the fetched clip. "" ?
unknown
4     # Cooperative re-enqueue: persist progress + next_poll_at, release the worker.
=== defs in job_worker ===
122: def _job_to_request(job: Dict[str, Any]):
145: def _run_claimed_job(job: Dict[str, Any], config: Any, db: Any) -> None:
=== get_highlights ===

```

27. user (2026-07-08T06:32:43.463Z)

[check-md-links] clean ? no broken internal links in live docs.

28. user (2026-07-08T06:32:56.849Z)

```
=== get_highlights / highlights endpoint search ===
backend/main.py:1041:@app.get("/api/highlights/{video_id}/{filename}")
backend/main.py:1042:def download_highlight(video_id: str, filename: str, request: Request):
backend/main.py:1099:def compile_highlights(req: CompileRequest, request: Request):
backend/orchestration.py:1228:     # #463: mirror the reel to the output bucket so
/api/highlights can 307 to a signed URL ? Cloud Run
backend/pipeline/stitcher/compiler.py:382:     def compile_highlights(
backend/reporting/harvest.py:181:def _highlights(
```

29. user (2026-07-08T06:33:11.532Z)

```
=== any get_highlights anywhere ===
exit=1
=== confirm 307 in download_highlight ===
12:     # #463: when the reel was mirrored to the output bucket, 307 to a short-lived signed URL
(bytes flow
14:     # exists/sign logic (with the active storage config) lives in
orchestration.signed_reel_url to keep
17:         signed = signed_reel_url(request.app.state.config, video_id, name)
18:         if signed:
19:             return RedirectResponse(signed, status_code=307)
21:         logger.warning("[highlights] signed-url path failed for %s; serving local: %s",
video_id, e)
```

30. assistant (2026-07-08T06:33:50.703Z)

Let me confirm `compute_video_id` is MD5-hex (underpins the "byte-identical" claim) and check the P2/DoD status consistency.

31. user (2026-07-08T06:33:54.680Z)

```
=== compute_video_id ===
3: `video_id` is the MD5 of the *whole* video file. Historically each endpoint
5: defined its own `get_file_md5` with a different read-chunk size (64KB / 8KB / 1MB).
23: def compute_video_id(filepath: str) -> str:
24:     """Return the MD5 hex digest of the whole file ? the canonical video_id.
29:     hasher = hashlib.md5()
33:     return hasher.hexdigest()
```

32. assistant (2026-07-08T06:34:25.395Z)

Let me check the actual Cloud Run Job vCPU/memory config to verify the cost-model machine spec.

33. user (2026-07-08T06:34:30.088Z)

```
=== cloud run job cpu/mem config ===
deploy/cloudrun/cloudbuild.yaml:26: machineType: E2_HIGHCPU_8     # the CUDA + micromamba image
is heavy; a bigger builder keeps it ~8?10 min
deploy/cloudrun/Dockerfile:4: # CPU/wall + egress are metered into one job ? D4). Built + pushed
+ verified by the owner on the
deploy/cloudrun/Dockerfile:12: #         validated on a real L4: cpu-decode F1 0.576 ? the torch-1.11
baseline 0.582) runs the WASB
deploy/cloudrun/Dockerfile:53: # PyNvVideoCodec dlopens. Harmless when decode_backend=cpu;
required for nvdec_resident.
deploy/cloudrun/Dockerfile:68: # (torch ?2 needs py?3.9; WASB HRNet ports clean ? cpu F1 0.576 ?
the torch-1.11 baseline 0.582) +
deploy/cloudrun/gen_service_yaml.py:23: * `run.googleapis.com/cpu-throttling: "false"` +
minScale 1 ? the in-process job worker drains
deploy/cloudrun/gen_service_yaml.py:24:     on a background thread; without always-allocated CPU
a 202'd compile stalls (README §3a).
deploy/cloudrun/gen_service_yaml.py:42: * `--memory` defaults to 8Gi (spec D2) ? compile still
pulls the gs:// source into the
```

```

deploy/cloudrun/gen_service_yaml.py:44: headroom; >8Gi would force 4 vCPU on Cloud Run for no
current need.
deploy/cloudrun/gen_service_yaml.py:116: run.googleapis.com/cpu-throttling: "false"
deploy/cloudrun/gen_service_yaml.py:127: cpu: "{args.cpu}"
deploy/cloudrun/gen_service_yaml.py:128: memory: {args.memory}
deploy/cloudrun/gen_service_yaml.py:182: p.add_argument("--cpu", type=int, default=2)
deploy/cloudrun/gen_service_yaml.py:184: "--memory",
deploy/cloudrun/post_deploy_smoke.ps1:14: # cpu path (parity) on the fast smoke clip:
deploy/cloudrun/post_deploy_smoke.ps1:15: ./post_deploy_smoke.ps1 -Video gs://khehsutra-rally-
corpus/videos/mahadevpura_smoke_180s.mp4 -Decode cpu
deploy/cloudrun/post_deploy_smoke.ps1:21: [ValidateSet("cpu", "nvdec_resident",
"env")][string]$Decode = "env",
deploy/cloudrun/post_deploy_smoke.ps1:44:# (exactly how prod dispatches behave).
"cpu"/"nvdec_resident" force it for this run only.
deploy/cloudrun/README.md:35: --cpu 8 --memory 32Gi \
deploy/cloudrun/README.md:45:- 8 vCPU / 32 GiB: x264 encode is CPU-side, so CPU headroom keeps
the paid GPU busy. `tmp` is
=== gcloud run jobs in deploy ===
deploy/cloudrun/gen_service_yaml.py:42: * `--memory` defaults to 8Gi (spec D2) ? compile still
pulls the gs:// source into the
deploy/cloudrun/gen_service_yaml.py:182: p.add_argument("--cpu", type=int, default=2)
deploy/cloudrun/gen_service_yaml.py:184: "--memory",
deploy/cloudrun/README.md:35: --cpu 8 --memory 32Gi \
deploy/cloudrun/README.md:46: **RAM-backed** ? the pulled input video lives in memory, so size
`--memory` for the largest
deploy/cloudrun/README.md:137: --cpu 2 --memory 4Gi --port 8080 --allow-unauthenticated \

```

34. assistant (2026-07-08T06:35:47.322Z)

I have completed verification. Here is my summary before the structured output.

What I verified

****Cost model (\$4.2) ? arithmetic is internally consistent and grounded:****

- L4 GPU \$0.00022404/s ? \$0.806/hr ?; L4 Job @ 8 vCPU/32 GiB ? \$0.000474/s ? \$1.705/hr ? (and 8 vCPU/32 GiB is the **actual** deployed config ? `deploy/cloudrun/README.md:35` `--cpu 8 --memory 32Gi`); g2-standard-4 \$0.872/hr ? \$637/mo ? ($\times 730$); spot \$0.523/hr ? \$382/mo ?; warm GPU Service \$1.256/hr ? \$917/mo ?; breakeven $0.872/1.705 = 51.1\%$?, spot 30.7% ? 31% ?; 2,600?4,300 runs/mo ?; per-run \$0.105 / \$0.148 / \$0.231 all recompute ?.
- Idle-GPU reasoning is sound (compute needs the whole file ? GPU idle during the whole upload).
- "same-region GCS?Cloud Run free/LAN-fast" is properly hedged `[researched]` (\$0 + \$6).

****Clean sub-dimensions:**** recommends-against-direction-#1 (Status/\$0/L5/\$8) ? clean; #521-open gaps flagged (\$6 P3 verdict-serialization `[verified]` and #513 threshold-forwarding `[verified]` ? both confirmed real in `backend/worker/\_\_main\_\_.py:277-284`) ? clean; win #1 credited to #521 and NOT implemented here (\$0/\$5/\$7/\$9) ? clean except one status-marker inconsistency (F5); `check\_md\_links` passes clean, PHASE_PLACEMENT.md refs are external GitHub URLs (correct ? not on master), runlog ref is a backtick code-span whose path resolves from the doc's dir ? clean; byte-identical `video\_id` claim verified (`compute\_video\_id` = MD5-hex; `gcs.\_md5\_hex` base64?hex); `submit\_time\_video\_id` (#484) / jobs.py submit-time-cache-trusts-md5 / placement.py `resolve\_phase\_placement` / D4·D7·D16 in README ? all verified accurate.

****Five findings**** (below), none change the doc's conclusion.

35. user (2026-07-08T06:36:06.192Z)

Structured output provided successfully